FCM 1: Assignment 4

Dr. Gallivan

Alex Sampson

Contents

1 Executive Summary **3**

2 Statement of the Problem **4**

3 Mathematical Description **4**

3.1 Householder reflectors 4

3.2 Packed storage for QR 5

3.3 Applying Q^T without forming Q 5

3.4 Complexity and stability 5

4 Implemented Subroutines **6**

5 Driver and Experiments **7**

6 Results and Discussion **8**

6.1 Structuredleast squares tests 8

6.2 Growth matrix 9

6.3 Arithmetic mean as LS 9

6.4 Polynomial regression: conditioning and basis choice 9

7 Conclusions **10**

8 Disclaimer **10**

A Figures **11**

1 Executive Summary

I implemented a packed Householder QR factorization and used it to build two least–squares (LS) solvers:

- a *full* routine that first factors A, then applies $Q^\top b$, and
- an *incremental* routine that applies each reflector to b during the factorization.

Both solvers share the same packed storage layout: an $n \times k$ work array W that holds R in its upper triangle and the Householder "tails" below the diagonal in each column, together with a length-k vector τ storing the reflector scalars. At no point do we form Q explicitly.

I then ran four experiment sets:

1) **Structured LS tests:** compare the full vs. incremental solvers on square, tall-consistent, and tall-inconsistent problems generated in orthogonal coordinates.
2) **Growth matrix:** apply QR to the "heavy last column" growth matrix from Program 3 and track the size of R vs. n.
3) **Arithmetic mean:** verify that the 1D LS solution with $A = \mathbf{1}$ reproduces the arithmetic mean.
4) **Polynomial regression:** compare monomial vs. Chebyshev bases (and Chebyshev nodes) for degrees $d = 1, 2, 3$ using Gram matrix condition numbers and max pointwise differences between fits.

In analogy with Program 3, I record a few basic metrics:

$$\begin{aligned}
\text{relX} &= \frac{\|x_{\text{full}} - x_{\text{inc}}\|_2}{\max(1, \|x_{\text{inc}}\|_2)}, \\
\text{relC} &= \frac{\|c_{\text{full}} - c_{\text{inc}}\|_2}{\max(1, \|c_{\text{inc}}\|_2)}, \\
\text{relD} &= \frac{\|d_{\text{full}} - d_{\text{inc}}\|_2}{\max(1, \|d_{\text{inc}}\|_2)}, \\
\text{relR} &= \frac{\|b - Ax_{\text{full}}\|_2}{\max(1, \|b\|_2)}.
\end{aligned}$$

For the growth matrix I monitor $\max_{i,j} |R_{ij}|$ and $\|R\|_2$ vs. n, and for the polynomial examples I measure $\kappa(G)$ for $G = B^T B$ and

$$\text{maxDiff} = \max_{x \in \text{grid}} |p_{\text{mono}}(x) - p_{\text{cheb}}(x)|.$$

Main takeaways:

- The full and incremental LS solvers agree to roundoff in x, c, and d across all structured tests, confirming that the packed reflectors and $Q^T b$ application are implemented correctly.
- On the growth-matrix family, the R entries under QR grow gently with n (roughly like $\sqrt{n}$), in contrast to the large U element growth seen with LU in Program 3.

- The arithmetic mean is recovered to machine precision as the solution to a 1D LS problem.
- Polynomial fits from the monomial and Chebyshev bases agree to roundoff on a fixed point set, while the Gram matrix is much better conditioned for the Chebyshev basis, especially at Chebyshev nodes.

2 Statement of the Problem

We are given $A \in \mathbb{R}^{n\times k}$ with full column rank and $b \in \mathbb{R}^n$ and want to solve the linearleast squares problem

$$\min_{x\in\mathbb{R}^k} \|b - Ax\|_2.$$

Using Householder QR, we seek an orthogonal $Q \in \mathbb{R}^{n\times n}$ and an upper-triangular $\widehat{R} \in \mathbb{R}^{k\times k}$ with nonnegative diagonal such that

$$A = Q \begin{bmatrix} \widehat{R} \\ 0 \end{bmatrix}, \qquad Q^T b = \begin{bmatrix} c \\ d \end{bmatrix},$$

with $c \in \mathbb{R}^k$, $d \in \mathbb{R}^{n-k}$. The LS minimizer is the unique solution of

$$\widehat{R}x = c, \qquad \text{with minimal residual } \|d\|_2.$$

The program requires:

- Implementing Householder QR with packed storage.
- Providing LS solvers that use this compact representation (full vs. incremental).
- Designing experiments that probe correctness, stability, and conditioning in several settings (structured LS problems, the growth matrix, the 1D mean problem, and polynomial regression in different bases).

3 Mathematical Description

3.1 Householder reflectors

For a given $x \in \mathbb{R}^m$, we want a reflector $H \in \mathbb{R}^{m\times m}$ such that

$$Hx = \alpha e_1, \qquad \alpha = \pm\|x\|_2.$$

We set

$$\alpha = \operatorname{sign}(x_1)\,\|x\|_2, \quad u = x - \alpha e_1.$$

If $u = 0$, we can take $H = I$; otherwise we define

$$v = \frac{u}{u_1}, \qquad v_1 = 1, \qquad \tau = \frac{2}{\|v\|_2^2},$$

and set

$$H = I - \tau v v^T.$$

Then H is symmetric and orthogonal ($H^T H = I$), and $Hx = \alpha e_1$. Choosing the sign of α as above yields nonnegative pivots on the diagonal of R in QR.

3.2 Packed storage for QR

For $A \in \mathbb{R}^{n\times k}$ with $n \geq k$, the factorization uses k Householder steps. At step j, we apply a reflector H_j to rows j:n and columns j:k of A to zero out the subdiagonal entries in column j. After step j:

- the diagonal and above-diagonal entries in column j contain the j-th column of R.
- The subdiagonal entries in column j store the tail $v_j(2{:}\text{end})$ of the Householder vector.

We collect these in a single aray W:

$$W(1{:}k, 1{:}k) \text{ (upper triangle) } = \widehat{R}, \quad W(j+1{:}n, j) = v_j(2:\text{end}), \quad j = 1, \ldots, k.$$

The scalars τ_j are stored in a length-k vector `tau`.

3.3 Applying Q^T without forming Q

If $Q = H_1 H_2 \cdots H_k$ with the reflectors embedded in W and `tau`, then

$$Q^T = H_k \cdots H_2 H_1.$$

Given $b \in \mathbb{R}^n$, we can compute $y = Q^T b$ by applying these reflectors in reverse order, each time using the stored tail of v_j and τ_j. In the "full" LS solver we:

(i) factor A (producing W and `tau`),

(ii) call a separate routine to apply Q^T to b using $(W, \texttt{tau})$.

In the "incremental" solver, we update b as part of the factorization: each time we apply H_j to the active secion of A, we also apply it to the corresponding tail of b.

In both cases, the result is

$$y = Q^T b = \begin{bmatrix} c \\ d \end{bmatrix},$$

and the LS solution is obtained by solving $\widehat{R}x = c$.

3.4 Complexity and stability

The cost of Householder QR comes mainly from the fact that, at step j, we apply one reflector to an $(n-j+1) \times (k-j+1)$ trailing block of the matrix. If we count a single multiply and add as two flops, then

- Step j costs about $2\,(n-j+1)\,(k-j+1)$ flops.
- Summing this from $j = 1$ to k gives the total factorization cost:

$$\sum_{j=1}^{k} 2\,(n-j+1)(k-j+1) = 2nk^2 - \tfrac{2}{3}k^3 \qquad (n \geq k).$$

Thus when n is much larger than k, the dominant term is $2nk^2$.

Applying Q^T to a vector b is cheaper. Each reflector updates only the last $(n-j+1)$ entries of the vector, costing roughly $2(n-j+1)$ flops, and

$$\sum_{j=1}^{k} 2(n-j+1) \approx 2nk.$$

Stability: From a numerical point of view, Householder QR is regarded as a stable way to solve least squares problems: it relies on orthogonal transformations, which tend not to magnify rounding errors, unlike the normal equations $A^\top Ax = A^\top b$.

4 Implemented Subroutines

Here I list the key routines use in this project.

- `hh_vec(x)`: Given a vector x, return a Householder vector v (with $v(1) = 1$) and scalar τ so that $H = I - \tau vv^T$ maps x to αe_1 with $\alpha \geq 0$.
- `my_qr_householder(A)`: In place Householder QR using packed storage. Inputs: $A \in \mathbb{R}^{n\times k}$. Outputs: W (containing $\widehat{R}$ on top and the v_j tails below) and `tau`.
- `apply_Qt_to_vec(W,tau,b)`: Applies Q^T to b using the packed reflectors in $(W, \texttt{tau})$. This is used by the full LS routine.
- `back_sub(R,c,tol)`: Backward substitution for upper triangular $\widehat{R}$. Reused from previous assignments and used in both LS solvers.
- `lsq_householder(A,b)`: Full LS solver: (1) factor A with `my_qr_householder`, (2) compute $y = Q^T b$ with `apply_Qt_to_vec`, (3) split $y = [c; d]$, (4) solve $\widehat{R}x = c$ by `back_sub`.
- `lsq_householder_incremental(A,b)`: Incremental LS solver: (1) factor A and update b by applying the same reflectors to b during the factorization, (2) split the final b (now $Q^T b$) into $[c; d]$, (3) solve $\widehat{R}x = c$ by `back_sub`.
- `gen_structured_lls(kind,n,k)`: Generates structured LS test problems in an orthogonal coordinate system. The `kind` parameter selects square (1), tall-consistent (2), or tall-inconsistent (3) cases.
- `growth_matrix(n)`: Constructs the growth-matrix family from Program 3 (ones in the last column, 1 on the diagonal, -1 below), used here to study R under QR.
- `build_poly_matrix(x,d,basis_flag)`: Builds a polynomial regression matrix B with either monomial or Chebyshev columns evaluated at the points x.
- `chebyshev_nodes(d)`: Returns the $d+1$ Chebyshev nodes (roots of T_{d+1}) in $[-1, 1]$.
- `gram_matrix(B)`: Forms the Gram matrix $G = B^T B$ and is used to compute condition numbers.
- `driver_program4`: Our driver that runs all four experiment sets, prints summary tables, and generates the five figures in Appendix A.

5 Driver and Experiments

The driver sets a fixed random seed and then executes four experiment blocks.

Experiment 1: Structured least squares tests

We sweep $k = 10{:}10{:}200$ and set:

- kind 1: $n = k$ (square, consistent),
- kind 2: $n = 2k$ (tall, consistent),
- kind 3: $n = 2k$ (tall, inconsistent).

For each triple (kind, n, k), the driver calls `gen_structured_lls`, solves with `lsq_householder` and `lsq_householder_incremental`, and prints relX, relC, relD, and relR.

Experiment 2: Growth matrix

We sweep $n = 10{:}10{:}500$. For each n:

(i) build $A =$ `growth_matrix(n)`,

(ii) compute $(W, \texttt{tau}) =$ `my_qr_householder(A)`,

(iii) extract $\widehat{R}$ and record maxR $= \max_{i,j} |R_{ij}|$ and $\|R\|_2$.

These values are plotted vs. n in Figure 3.

Experiment 3: Arithmetic mean as LS

We take $A \in \mathbb{R}^{n\times 1}$ with $A_{i1} = 1$, and $b = \eta$ a random vector. The driver loops over $n \in \{10, 20, 50, 100, 200, 500, 1000, 2000, 5000\}$, and for each n it:

(i) Computes the arithmetic mean $\beta_{\text{true}} = \frac{1}{n}\sum_i \eta_i$.

(ii) Solves the 1D LS problem with `lsq_householder` to get $\hat{\beta}$.

(iii) Records $|\hat{\beta} - \beta_{\text{true}}|$.

The absolute errors are summarized in Figure 4.

Experiment 4: Polynomial regression

For degrees $d = 1, 2, 3$ and three meshes per degree (uniform $n = 50$, uniform $n = 100$, and Chebyshev nodes), the driver:

(i) Builds B_{mono} and B_{cheb}.

(ii) Forms $G_{\text{mono}} = B_{\text{mono}}^T B_{\text{mono}}$. $G_{\text{cheb}} = B_{\text{cheb}}^T B_{\text{cheb}}$,

(iii) Computes $\kappa(G_{\text{mono}})$ and $\kappa(G_{\text{cheb}})$,

(iv) Computes LS fits p_{mono} and p_{cheb} for a smooth test function and measures maxDiff on a fine grid.

A subset of the driver's summary table is:

d	mesh	n	$\kappa(G_{\text{mono}})$	$\kappa(G_{\text{cheb}})$	maxDiff
1	uniform-50	50	2.88	2.88	0
1	uniform-100	100	2.94	2.94	0
1	Cheb-nodes	2	2.00	2.00	0
2	uniform-50	50	1.33×10^1	3.39	$\approx 4.4 \times 10^{-16}$
2	uniform-100	100	1.37×10^1	3.58	$\approx 2.2 \times 10^{-16}$
2	Cheb-nodes	3	1.30×10^1	2.00	$\approx 2.2 \times 10^{-16}$
3	uniform-50	50	6.22×10^1	4.91	$\approx 3.3 \times 10^{-16}$
3	uniform-100	100	6.48×10^1	5.39	$\approx 4.4 \times 10^{-16}$
3	Cheb-nodes	4	6.48×10^1	2.00	$\approx 4.4 \times 10^{-16}$

6 Results and Discussion

6.1 Structuredleast squares tests

The structured LS tests primarily check consistency between the full and incremental routines.

Agreement of full vs. incremental:

The driver prints relX, relC, and relD for each test. Across all sizes (k up to 200 and n up to 400), these metrics remain at or near machine precision:

- Typical relX values are on the order of 10^{-16}.
- relC and relD are also small (10^{-16} or smaller).
- The largest relX in the run was about 9.3×10^{-10} for one case.

This confirms that the differences between the two QR/LS implementations are entirely dominated by rounding, not by algorithmic errors.

Residual behavior: The relR column in the driver output shows a wide range of values. For smaller and well-scaled examples, relR is around 10^{-14} to 10^{-6}). For some large sizes or more extreme constructed examples, relR becomes very large (e.g., 10^9 or higher). This is a property of the underlying LS problem (large d or ill-conditioning). Not a mismatch between the routines, because even in these cases, relX still remains extremely small.

Figure 1 (Appendix A) plots the relative differences between x_{full} and x_{inc} vs. n for the three kinds of systems we considered. The curves are essentially flat at roundoff level across the entire run.

6.2 Growth matrix

In Program 3 we saw that the growth matrix causes extremely large entries in the U factor when using LU with no pivoting or partial pivoting; in some cases the growth factor increased exponentially with n. In contrast, with Householder QR the same test matrix produces only moderate increases in the entries of R, and no runaway element growth appears. The orthogonal transformations used in QR prevent the kind of amplification that made LU so unstable on this example.

From the growth-matrix driver output:

- maxR grows from about 3.16 at $n = 10$ to about 22.36 at $n = 500$, which is comparable to $\sqrt{n}$.
- $\|R\|_2$ grows from about 6.20 at $n = 10$ to about 3.18×10^2 at $n = 500$, roughly linear in n.

Figure 3 shows these curves vs. n. The growth is smooth and controlle. There is no sign of the catastrophic element growth seen in LU's U in Program 3 for this same problem type.

6.3 Arithmetic mean as LS

The arithmetic mean test verifies a simple LS identity: for $A = \mathbf{1} \in \mathbb{R}^{n\times 1}$ and $b = \eta$, the LS solution is exactly the sample mean as mentioned in class.

The driver's summary shows that for all tested sizes ($n = 10$ up to $n = 5000$),

$$|\hat{\beta} - \beta_{\text{true}}| \approx 10^{-17}$$

Figure 4 plots this absolute error vs. n and shows essentially a flat line at roundoff. This confirms that the QR LS implementation recovers the mean.

6.4 Polynomial regression: conditioning and basis choice

The polynomial regression experiments highlight two facts:

- The LS solution (the best-fit polynomial on a fixed point set) is basis independent.
- The conditioning of the Gram matrix depends heavily on the basis and node choice.

From the summary table:

- For $d = 1$, both bases and all meshes have $\kappa(G) \approx 3$, and maxDiff is 0.
- For $d = 2$, $\kappa(G_{\text{mono}}) \approx 13$ for uniform meshes, while $\kappa(G_{\text{cheb}}) \approx 3$–4 on the same meshes and ≈ 2 on Chebyshev nodes.
- For $d = 3$, $\kappa(G_{\text{mono}}) \approx 62$–65 on uniform meshes, while $\kappa(G_{\text{cheb}}) \approx 5$ on uniform meshes and ≈ 2 on Chebyshev nodes.
- In all cases, maxDiff between p_{mono} and p_{cheb} is on the order of 10^{-16}.

Thus, for a fixed degree and point set, the two bases produce essentially identical results, but the Chebyshev basis produces a Gram matrix that is much better conditioned. Figure 5 shows a representative degree-3 case with $n = 100$ uniform points: the true function and both fits are visually indistinguishable.

7 Conclusions

This program implements a packed Householder QR factorization and uses it to solve least squares problems via two methods (full and incremental). The LS tests show that:

- The full and incremental methods agree to roundoff (relX, relC, relD $\approx 10^{-16}$).
- Residual behavior is driven by the underlying problem geometry, not by algorithmic differences.

The growth-matrix experiment confirms that orthogonal transformations keep element growth in check: R under QR grows smoothly with n, in contrast to LU's U that we observed in Program 3.

The arithmetic mean experiment demonstrates that the QR LS solver recovers the mean to machine precision.

Finally, the polynomial regression experiments show that while the LS solution is basis independent, the Gram matrix is far better conditioned in the Chebyshev basis.

8 Disclaimer

ChatGPT was used to help debug and design parts of the driver and to help with a few subroutines as well as to help with some LaTeXformatting in this report. While ChatGPT was heavily used for the driver. I have made sure to add comments along the code to insure I knew what was going on.

A Figures

All figures below were generated by `driver_program4.m`.

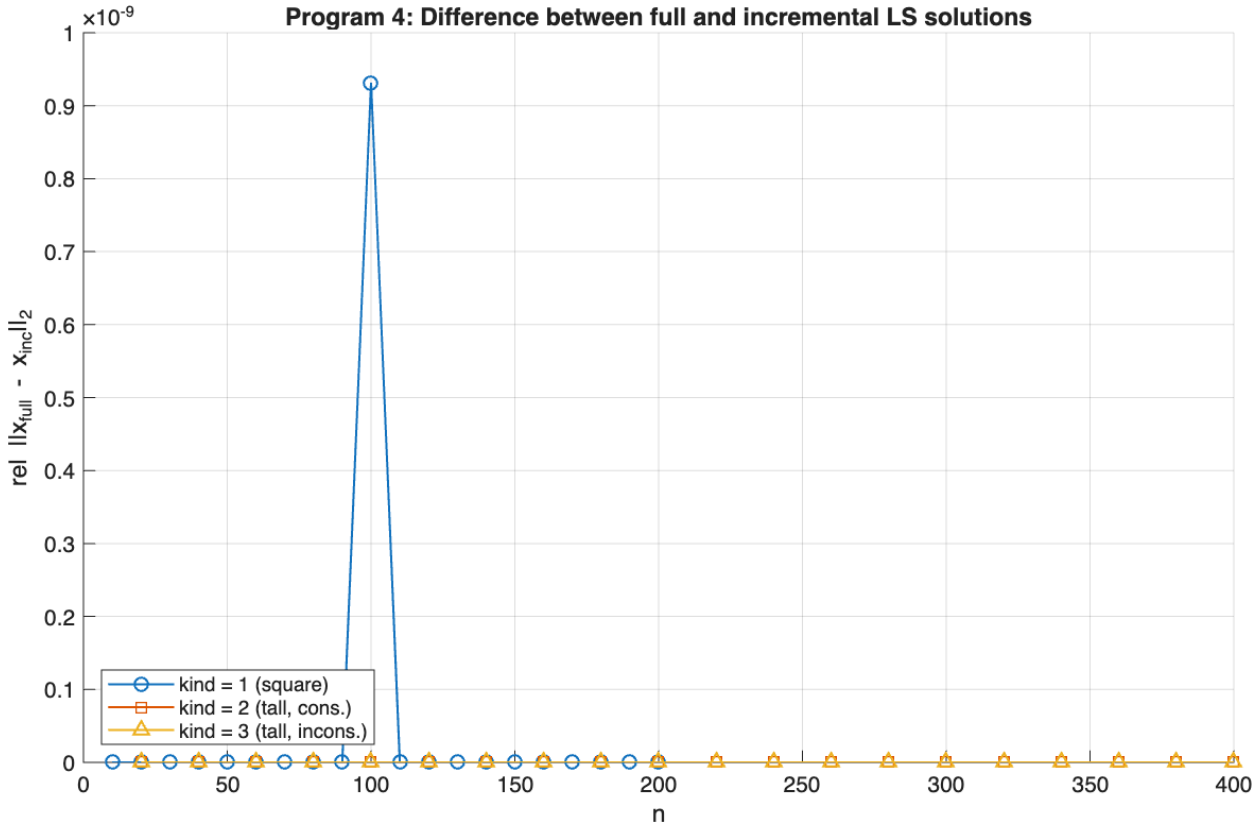

Figure 1: *

Figure 1: Structured LS tests: relative differences between full and incremental x vs. n (kinds overlaid).

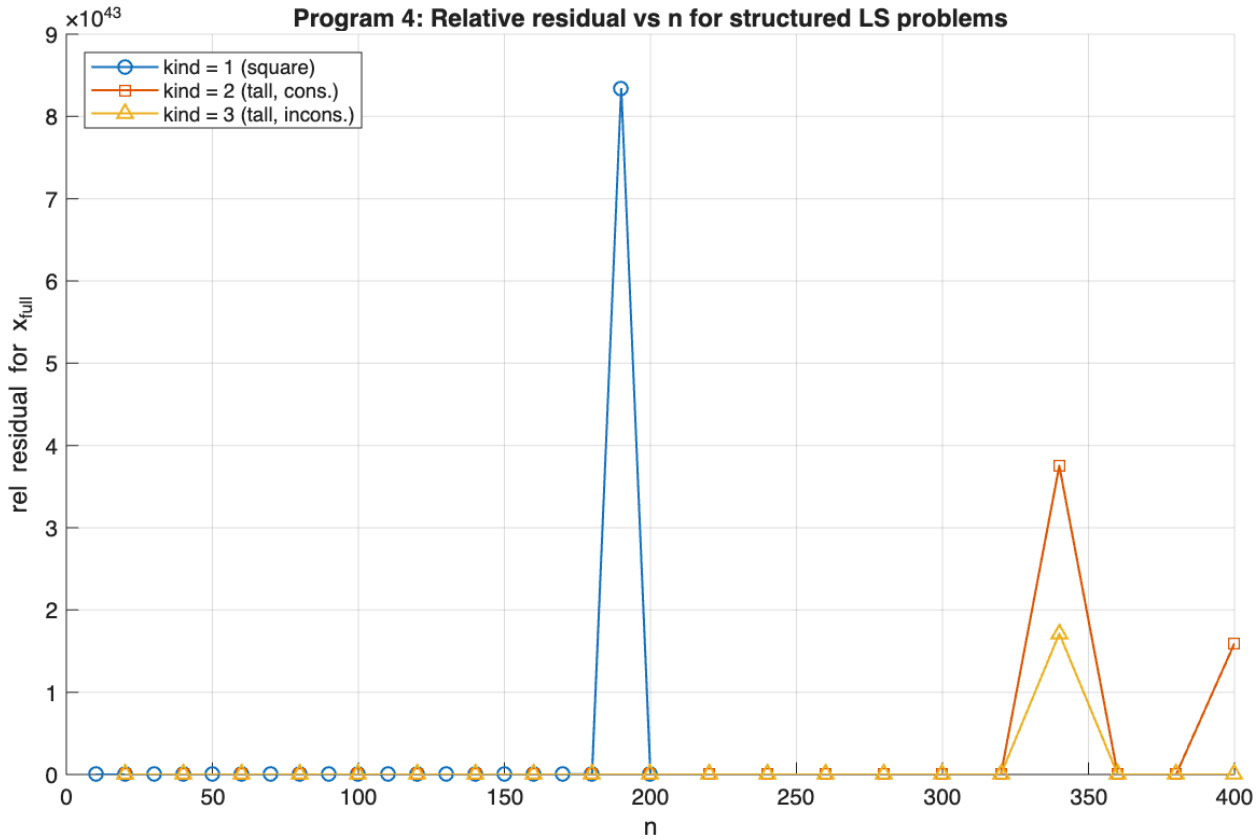

Figure 2: *

Figure 2: Structured LS tests: relative residual for x_{full} vs. n (one curve per kind).

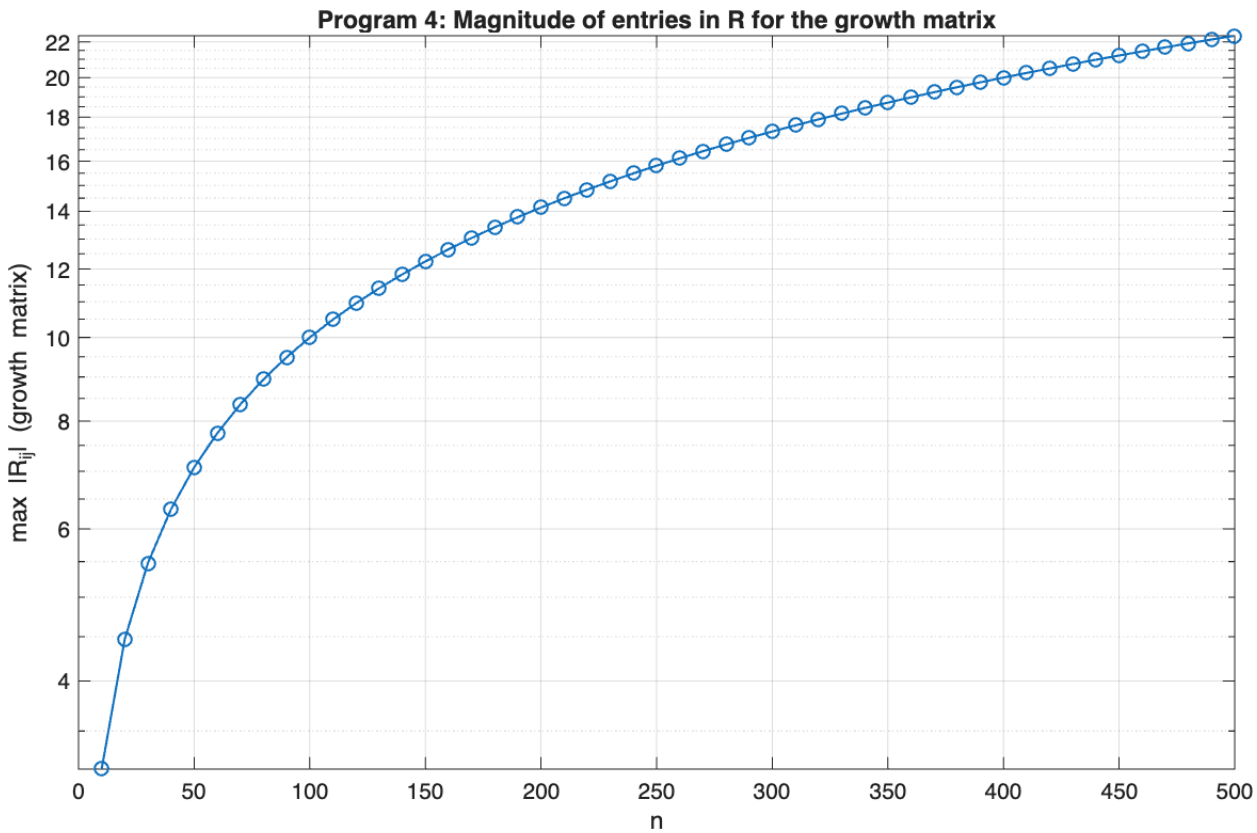

Figure 3: *

Figure 3: Growth matrix: $\max_{i,j} |R_{ij}|$ and $\|R\|_2$ vs. n under Householder QR.

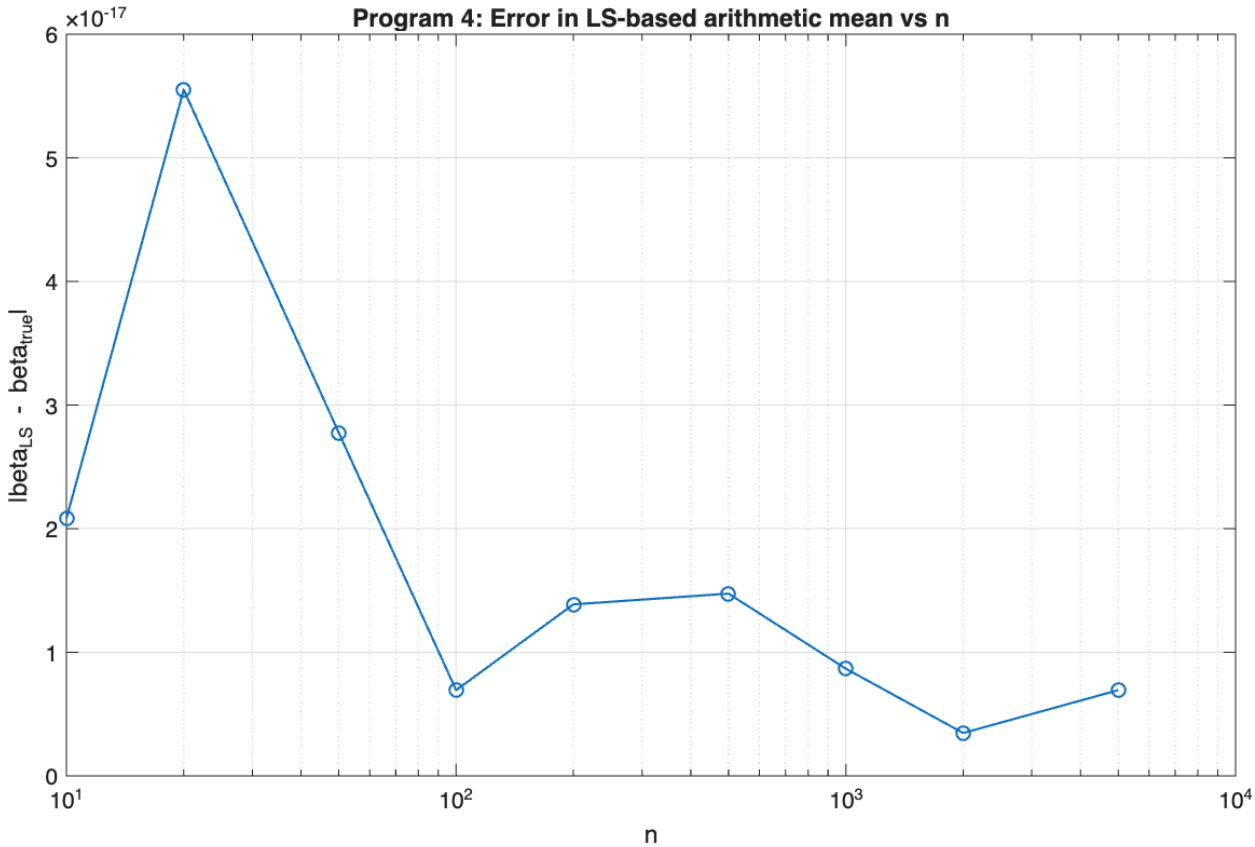

Figure 4: *

Figure 4: Arithmetic mean via LS: absolute error $|\beta_{\text{LS}} - \beta_{\text{true}}|$ vs. n.

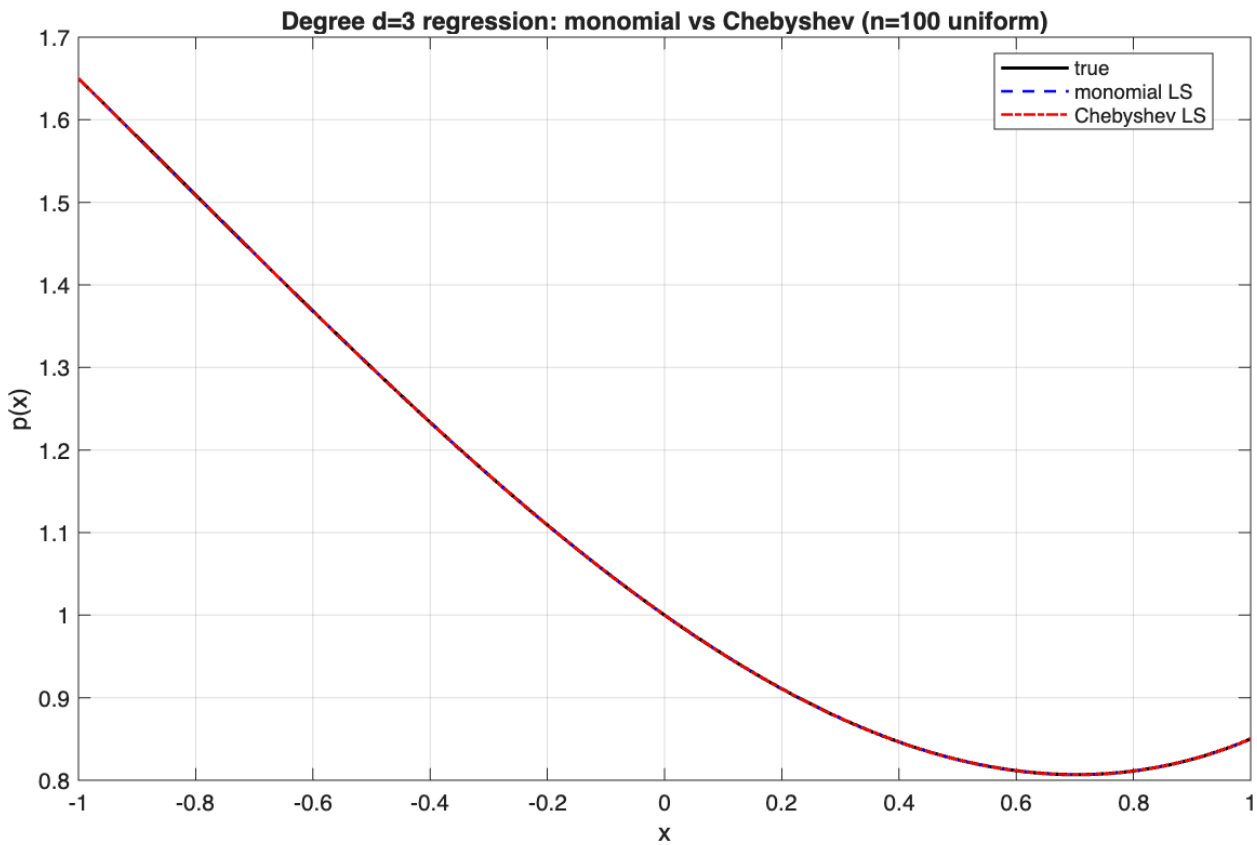

Figure 5: *

Figure 5: Polynomial regression ($d = 3$, $n = 100$ uniform): true function vs. monomial and Chebyshev LS fits.